PHASE 5 — CRYPTOGRAPHIC DISCOVERY, CBOM & ASSET INVENTORY

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 5 of 22

Document Type: Technical Research & Engineering Handbook

Status: Core Discovery Architecture

Table of Contents

22. Dependency Discovery
23. Repository Discovery
24. Binary Discovery
25. Container Discovery
26. Firmware Discovery
27. Infrastructure Discovery
28. Cloud Discovery
29. Static Analysis
30. Regex-Based Detection
31. AST-Based Detection
32. Semantic Code Analysis
33. Dependency Analysis
34. Binary Analysis
35. String Analysis
36. Symbol Analysis
37. Library Fingerprinting
38. Certificate Parsing
39. TLS Discovery
40. Runtime Discovery
41. Evidence Collection
42. Detection Confidence
43. False Positives
44. False Negatives
45. Asset Deduplication
46. Asset Identity
47. Cryptographic Lineage
48. Version Tracking
49. Ownership Mapping
50. Data Classification
51. CBOM Generation Pipeline
52. Example CBOM
53. Discovery Architecture
54. Continuous Discovery

---

1. Purpose of This Phase

The previous phases established the theoretical foundation:

```
Phase 1
Cryptography
       ↓
Phase 2
Quantum Threat
       ↓
Phase 3
PQC
       ↓
Phase 4
Enterprise Cryptographic Ecosystem
```

Now we begin designing the actual heart of ECDAT:

Cryptographic Discovery

The NTRO problem statement explicitly asks for a system capable of scanning:

- Source code repositories
- Binaries
- Libraries

- Container images

and producing a standardized inventory of cryptographic assets.

This means ECDAT needs a discovery engine capable of answering:

> **What cryptography exists, where does it exist, how is it being used, what depends on it, and what evidence proves that it exists?**

That inventory becomes the foundation for every later component:

```
Discovery
   ↓
CBOM
   ↓
Risk Analysis
   ↓
PQC Recommendation
   ↓
Migration Planning
```

---

2. The Discovery Problem

Suppose ECDAT scans a repository and finds:

```
RSA.generate(2048)
```

That is easy.

But real enterprise systems may look like:

```
Application
   ↓
Framework
   ↓
Security Provider
   ↓
Native Library
   ↓
OpenSSL
   ↓
RSA
```

The source code may never explicitly mention RSA.

Similarly, a container may contain:

```
Application
   ↓
libssl.so
   ↓
OpenSSL
   ↓
TLS
   ↓
ECDHE
```

A certificate may reveal:

```
Public Key:
EC P-256

Signature:
SHA256withECDSA
```

A Kubernetes deployment might use:

```
Ingress
   ↓
TLS Secret
   ↓
Certificate
   ↓
Private Key
```

Therefore, discovery must happen at multiple layers.

---

3. What ECDAT Must Discover

A comprehensive inventory should identify:

Algorithms

```
RSA
ECDSA
ECDH
DH
AES
```

```
3DES
SHA-1
SHA-2
SHA-3
ChaCha20
ML-KEM
ML-DSA
SLH-DSA
```

Keys

- Public keys
- Private key references
- Symmetric key references
- Key IDs
- Key stores
- HSM-backed keys
- Cloud KMS keys

Certificates

- X.509
- TLS certificates
- Device certificates
- Code-signing certificates
- CA certificates

Protocols

- TLS
- SSH
- IPsec
- VPN
- mTLS
- Secure boot
- Code signing

Libraries

- OpenSSL
- BoringSSL
- LibreSSL
- Bouncy Castle
- libsodium
- mbed TLS
- wolfSSL

Infrastructure

- HSM
- KMS
- Certificate authorities
- API gateways
- Service meshes
- Load balancers

---

4. Cryptographic Discovery vs Vulnerability Scanning

These are not the same thing.

A vulnerability scanner asks:

> Is something vulnerable?

ECDAT first asks:

> What cryptographic assets exist?

Example:

```
RSA-2048
```

A conventional scanner might say:

```
No known critical vulnerability.
```

ECDAT might say:

```
Quantum Vulnerability:
High
```

```
Reason:
RSA relies on integer factorization.

Data Lifetime:
20 years

Migration Complexity:
High

Recommendation:
Plan PQC migration.
```

Therefore:

```
Vulnerability Scanning
        ≠
Cryptographic Discovery
```

ECDAT can use vulnerability intelligence, but its primary mission is **cryptographic visibility and quantum readiness**.

---

5. What Is a CBOM?

CBOM stands for:

Cryptographic Bill of Materials

It is conceptually similar to a Software Bill of Materials (SBOM), but focuses specifically on cryptographic components and dependencies.

An SBOM might say:

```
Application
↓
OpenSSL 3.x
```

A CBOM goes deeper:

```
Application
↓
OpenSSL
↓
TLS
```

```
↓
ECDHE
↓
ECDSA
↓
P-256
↓
Certificate
```

The CBOM therefore represents the organization's cryptographic composition.

---

6. Why CBOM Is Necessary

Without an inventory, organizations may not know:

- Where RSA exists
- Where ECC exists
- Which applications use weak algorithms
- Which certificates are expiring
- Which systems depend on vulnerable libraries
- Which applications need PQC migration

A CBOM provides a structured representation.

Conceptually:

```
Enterprise
   ↓
Cryptographic Assets
   ↓
Relationships
   ↓
Risk
   ↓
Migration
```

---

7. CBOM vs SBOM

Property	SBOM	CBOM

Primary Focus	Software components	Cryptographic components
Libraries	Yes	Yes
Algorithms	Sometimes	Core focus
Certificates	Usually no	Yes
Keys	Usually no	References/metadata
Protocols	Limited	Important
Crypto APIs	No	Important
PQC Readiness	No	Yes
Quantum Risk	No	Core capability

ECDAT can use SBOM information as an input but must build additional cryptographic intelligence.

---

8. CBOM vs Asset Inventory

An asset inventory answers:

> What systems do we have?

A CBOM answers:

> What cryptography exists inside those systems?

For example:

```
Asset Inventory

Payment API
Server
Database
Load Balancer
```

CBOM:

```
Payment API
│
├── TLS
│   ├── ECDHE
│   ├── AES-256-GCM
│   └── ECDSA P-256
│
├── Database
│   └── AES-256
│
└── Signing
    └── RSA-2048
```

ECDAT should combine both perspectives.

---

9. CBOM vs Vulnerability Management

A vulnerability management system may focus on:

```
CVE
CVSS
Exploitability
Patch
```

ECDAT focuses on:

```
Algorithm
Key
Certificate
```

```
Protocol
Crypto Library
Usage
Quantum Vulnerability
Migration
```

These systems complement each other.

For example:

```
OpenSSL
  │
  ├── CVE Risk
  │
  └── Quantum Crypto Risk
```

An enterprise needs both.

---

10. CBOM Standards and Formats

ECDAT should avoid creating a completely proprietary format if standardized formats can represent relevant information.

Important ecosystems include:

- CycloneDX
- SPDX
- CBOM-related specifications and extensions

The goal should be interoperability.

For example:

```
ECDAT
  ↓
CBOM
  ↓
Export
  ↓
JSON
  ↓
Other Security Tools
```

---

11. CycloneDX

CycloneDX is a widely used SBOM format.

It can represent:

- Components
- Dependencies
- Services
- Metadata
- Vulnerabilities
- Properties

ECDAT can use CycloneDX as a foundation and extend it with cryptographic properties where appropriate.

Example conceptual structure:

```
{
  "component": {
    "name": "payments-api",
    "type": "application"
  },
  "crypto": {
    "algorithm": "ECDSA",
    "parameter": "P-256",
    "function": "signature"
  }
}
```

The exact schema should follow the relevant current specification rather than inventing incompatible fields.

---

12. SPDX

SPDX is another major software bill-of-materials standard.

It provides structured information about:

- Packages
- Files
- Relationships
- Licenses
- Dependencies

ECDAT can use SPDX-derived dependency information to identify where cryptographic libraries originate.

For example:

```
Application
↓
Package
↓
Dependency
↓
Crypto Library
```

---

13. ECDAT's Internal CBOM Model

Regardless of external export format, ECDAT should maintain an internal normalized model.

A conceptual model:

```
CryptoAsset
│
├── Identity
├── Type
├── Algorithm
├── Function
├── Location
├── Evidence
├── Dependencies
├── Owner
├── Data Context
├── Quantum Status
├── Risk
└── Migration
```

This allows ECDAT to support multiple output formats.

---

14. Cryptographic Asset Types

ECDAT should define asset categories.

Algorithm

```
RSA
AES
ECDSA
```

Key

```
KMS Key
HSM Key
Certificate Public Key
```

Certificate

```
X.509
```

Protocol

```
TLS
SSH
IPsec
```

Library

```
OpenSSL
BoringSSL
```

API

```
EVP
JCA
.NET Crypto
```

Service

```
KMS
HSM
Certificate Authority
```

---

15. Algorithm Discovery

Algorithm discovery can happen through multiple methods.

Direct source match

```
RSA
AES
ECDSA
```

API mapping

```
RSA.generate()
↓
RSA
```

Library metadata

```
OpenSSL
↓
Crypto capability mapping
```

Certificate parsing

```
Public Key Algorithm:
ECDSA
```

Protocol inspection

```
TLS handshake:
ECDHE
```

ECDAT should combine evidence instead of relying on one method.

---

16. Key Discovery

ECDAT should distinguish between:

Key existence

```
KMS key ID detected
```

and:

Key material

```
Private key bytes
```

The first is useful.

The second is extremely sensitive.

A secure design should generally avoid extracting private key material.

Instead:

```
Key ID
Algorithm
Location
Owner
Key State
Rotation Policy
```

should be collected where authorized.

---

17. Certificate Discovery

Certificate parsing is highly structured.

ECDAT can extract:

```
Subject
Issuer
Serial
Validity
Public Key
Key Size
Signature Algorithm
SAN
Key Usage
Extended Key Usage
Chain
```

Example:

```
Certificate
├── Subject: api.example
├── Public Key: ECDSA P-256
├── Signature: SHA256withECDSA
├── Valid From: ...
└── Valid Until: ...
```

---

18. Protocol Discovery

ECDAT should identify protocols such as:

```
TLS
SSH
IPsec
mTLS
S/MIME
DNSSEC
```

Protocol discovery should then expose cryptographic parameters.

For TLS:

```
Protocol
↓
Version
↓
Cipher Suite
↓
Key Exchange
↓
Certificate
↓
Signature
```

---

19. Library Discovery

Library discovery should capture:

```
Name
Version
Vendor
Source
Package Manager
Binary Path
Application
Known Algorithms
```

Example:

```
OpenSSL
Version 3.x
Installed:
Production Container
```

The version matters because cryptographic capabilities can change between releases.

---

20. API Discovery

API discovery maps code to cryptographic functionality.

Example:

```
Cipher.getInstance("AES/GCM/NoPadding")
```

becomes:

```
Algorithm:
AES

Mode:
GCM

Function:
Encryption

Language:
Java

Evidence:
API invocation
```

This is much more valuable than storing a raw text match.

---

21. Configuration Discovery

Cryptographic configuration may be found in:

```
YAML
JSON
XML
INI
```

```
TOML
ENV
Shell scripts
Infrastructure-as-Code
```

Example:

```
tls:
  version: TLSv1.3
  certificate: /certs/server.pem
```

ECDAT should parse configuration semantically.

---

22. Dependency Discovery

Suppose:

```
Application
↓
Framework
↓
Library A
↓
Library B
↓
OpenSSL
```

ECDAT needs to preserve this dependency chain.

Why?

Because migrating the algorithm may require updating:

```
Library B
```

rather than changing the application itself.

---

23. Repository Discovery

Repository scanning should include:

```
Repository
├── Source
├── Build files
```

```
    ├── Dependencies
    ├── Configuration
    ├── Certificates
    ├── Scripts
    ├── Infrastructure
    └── Documentation
```

Important files include:

```
package.json
pom.xml
build.gradle
requirements.txt
Cargo.toml
go.mod
Dockerfile
terraform files
Kubernetes manifests
```

---

24. Binary Discovery

Binaries may contain cryptographic dependencies invisible from source.

ECDAT can inspect:

- ELF
- PE
- Mach-O
- WASM where relevant

Potential evidence:

```
Imports
Exports
Symbols
Strings
Sections
Linked Libraries
Embedded Certificates
Embedded Public Keys
```

---

25. Container Discovery

Container scanning should inspect:

```
Image
├── Layers
├── OS Packages
├── Shared Libraries
├── Application
├── Configurations
├── Certificates
└── Metadata
```

A cryptographic library may come from a base image.

Therefore, ECDAT should identify provenance.

---

26. Firmware Discovery

Firmware analysis may include:

```
Binary
↓
Strings
↓
Symbols
↓
Embedded Certificates
↓
Public Keys
↓
Crypto Library Fingerprints
```

Firmware is often difficult because symbols may be stripped.

This makes confidence scoring essential.

---

27. Infrastructure Discovery

Infrastructure sources may include:

```
Nginx
Apache
HAProxy
F5
Cloud Load Balancers
```

```
VPN Gateways
Firewalls
Service Mesh
Kubernetes
```

ECDAT should collect configuration and runtime metadata where authorized.

---

28. Cloud Discovery

Cloud APIs can provide:

```
KMS Keys
Certificates
TLS Policies
HSMs
Secrets
Encryption Settings
```

ECDAT should eventually support provider-specific connectors.

The architecture should use a common abstraction:

```
Cloud Connector
      ↓
Normalized Crypto Asset
```

---

29. Static Analysis

Static analysis examines artifacts without executing them.

ECDAT can combine:

```
Regex
+
AST
+
Dependency Analysis
+
Symbol Analysis
+
Binary Analysis
```

Static analysis should be the foundation of repository scanning.

---

30. Regex-Based Detection

Regex is the simplest detection mechanism.

Example:

```
RSA|ECDSA|AES|SHA256|OpenSSL
```

Advantages:

- Fast
- Easy
- Broad initial coverage

Disadvantages:

- False positives
- False negatives
- No semantic understanding

Example:

```
RSA = Risk Score Algorithm
```

A naive regex scanner might incorrectly classify it as RSA cryptography.

Therefore, regex should be only the first layer.

---

31. AST-Based Detection

An Abstract Syntax Tree represents source code structurally.

Example:

```
AES.new(key)
```

can be represented conceptually as:

```
Call
├── Function: AES.new
└── Arguments
    └── key
```

AST analysis can determine:

- Actual function calls
- Imports

- Arguments
- Control flow context

This significantly improves detection accuracy.

---

32. Semantic Code Analysis

Consider:

```
security.encrypt(payload)
```

The name does not reveal the algorithm.

Semantic analysis might trace:

```
security.encrypt
↓
CryptoManager.encrypt
↓
AES-GCM implementation
```

This is much harder.

Potential techniques include:

- Call graph analysis
- Data-flow analysis
- Type analysis
- Symbol resolution
- LLM-assisted reasoning

This will become a major part of Phase 8.

---

33. Dependency Analysis

Dependency analysis determines:

```
Who depends on whom?
```

Example:

```
payments-api
↓
Spring Security
↓
```

```
Bouncy Castle
↓
Crypto Provider
```

This allows ECDAT to identify indirect cryptographic usage.

---

34. Binary Analysis

Binary analysis may involve:

```
Executable
↓
Headers
↓
Imports
↓
Symbols
↓
Strings
↓
Libraries
↓
Crypto Fingerprints
```

Tools may include:

- `readelf`
- `objdump`
- `nm`
- `strings`
- platform-specific binary inspection tools

More advanced analysis can use reverse-engineering frameworks.

---

35. String Analysis

Strings can reveal:

```
RSA
ECDSA
AES
BEGIN CERTIFICATE
BEGIN PUBLIC KEY
```

```
OpenSSL
TLS
```

But string matches are weak evidence.

ECDAT should label them:

```
Evidence Type:
String Match
Confidence:
Low/Medium
```

rather than treating them as definitive.

---

36. Symbol Analysis

Symbols can provide stronger evidence.

For example:

```
RSA_public_encrypt
EVP_EncryptInit_ex
EC_KEY_new
```

These symbols can indicate cryptographic library usage.

However:

- symbols may be stripped,
- symbols may be statically linked,
- symbols may be renamed,
- wrappers may hide the original calls.

Therefore, multiple detection methods are necessary.

---

37. Library Fingerprinting

ECDAT can identify libraries through:

- Package metadata
- File hashes
- Symbols
- Version strings
- Known signatures

- Dependency manifests

For example:

```
Binary
↓
Known crypto-library fingerprint
↓
OpenSSL
↓
Version estimate
```

The system should distinguish exact version from inferred version.

---

38. Certificate Parsing

Certificates provide extremely valuable structured evidence.

ECDAT can use standard certificate parsing libraries to extract:

```
Algorithm
Key Type
Key Size
Signature
Validity
Issuer
Subject
Extensions
```

This should be deterministic rather than AI-generated.

---

39. TLS Discovery

TLS can be discovered through:

Configuration

```
nginx.conf
```

Certificate

```
server.pem
```

Runtime endpoint inspection

```
TLS handshake
```

Library analysis

```
OpenSSL
```

ECDAT should correlate these.

Example:

```
Endpoint
↓
TLS 1.3
↓
ECDHE
↓
AES-256-GCM
↓
ECDSA P-256 Certificate
```

---

40. Runtime Discovery

Runtime analysis can observe actual cryptographic behavior.

Possible signals:

```
Process
↓
Loaded Library
↓
Crypto Function
↓
Network Connection
```

Runtime data can validate static findings.

For example:

```
Static:
OpenSSL present

Runtime:
OpenSSL actually loaded
```

```
Confidence:
High
```

---

41. Evidence Collection

Every finding should contain evidence.

Example:

```
{
  "finding": "ECDSA P-256",
  "source": "certificate",
  "location": "/etc/tls/server.pem",
  "evidence": {
    "public_key_algorithm": "EC",
    "curve": "P-256",
    "signature_algorithm": "ECDSA"
  }
}
```

Evidence is critical for explainability.

---

42. Detection Confidence

ECDAT should calculate confidence based on evidence.

Example:

```
Direct API:
0.99

Certificate parsing:
0.99

Binary symbol:
0.93

Dependency inference:
0.85

String match:
0.60
```

```
LLM inference:
0.70
```

These are illustrative values, not universal standards.

The exact confidence model should be calibrated against test datasets.

---

43. False Positives

False positive:

> Scanner reports cryptography where none exists.

Example:

```
Class name:
RSAConfiguration
```

but no cryptographic functionality exists.

False positives reduce trust.

Therefore:

```
Detection
+
Context
+
Evidence
```

must be used.

---

44. False Negatives

False negative:

> Cryptography exists but ECDAT fails to detect it.

Example:

```
CustomCrypto.encrypt()
```

where the underlying implementation uses RSA.

False negatives are particularly dangerous because they create a false sense of security.

ECDAT should optimize for both.

---

45. Asset Deduplication

The same asset may appear many times.

Example:

```
OpenSSL
```

may be discovered through:

- Package manager
- Binary
- Container
- Source dependency
- Runtime process

ECDAT should consolidate them.

Instead of:

```
5 findings
```

represent:

```
1 underlying library
+
5 evidence sources
```

This is much cleaner.

---

46. Asset Identity

ECDAT needs a stable identity model.

Example:

```
Asset ID:
CRYPTO-019283
```

The ID should remain stable even when new evidence appears.

Possible identity inputs:

```
Type
Canonical Name
Version
```

```
Location
Hash
Provider
```

---

47. Cryptographic Lineage

Lineage describes where an asset came from.

Example:

```
Base Image
   ↓
OpenSSL Package
   ↓
Container
   ↓
Application
   ↓
Production Service
```

ECDAT can answer:

> "Why does this application contain this cryptographic library?"

This is extremely valuable for remediation.

---

48. Version Tracking

Cryptographic inventory must be temporal.

Example:

```
August:
OpenSSL 3.0

September:
OpenSSL 3.1

October:
OpenSSL 3.2
```

ECDAT should preserve history.

This enables:

- Trend analysis
- Migration tracking
- Regression detection
- Compliance reporting

---

49. Ownership Mapping

Each asset should ideally connect to:

```
Asset
↓
Application
↓
Team
↓
Business Unit
↓
Owner
```

This allows automated task generation.

Example:

```
Critical PQC migration
↓
Payments Team
↓
Security Owner
```

---

50. Data Classification

ECDAT should capture the data protected by the asset.

For example:

```
Asset:
RSA-2048

Protects:
Customer Identity Records

Classification:
```

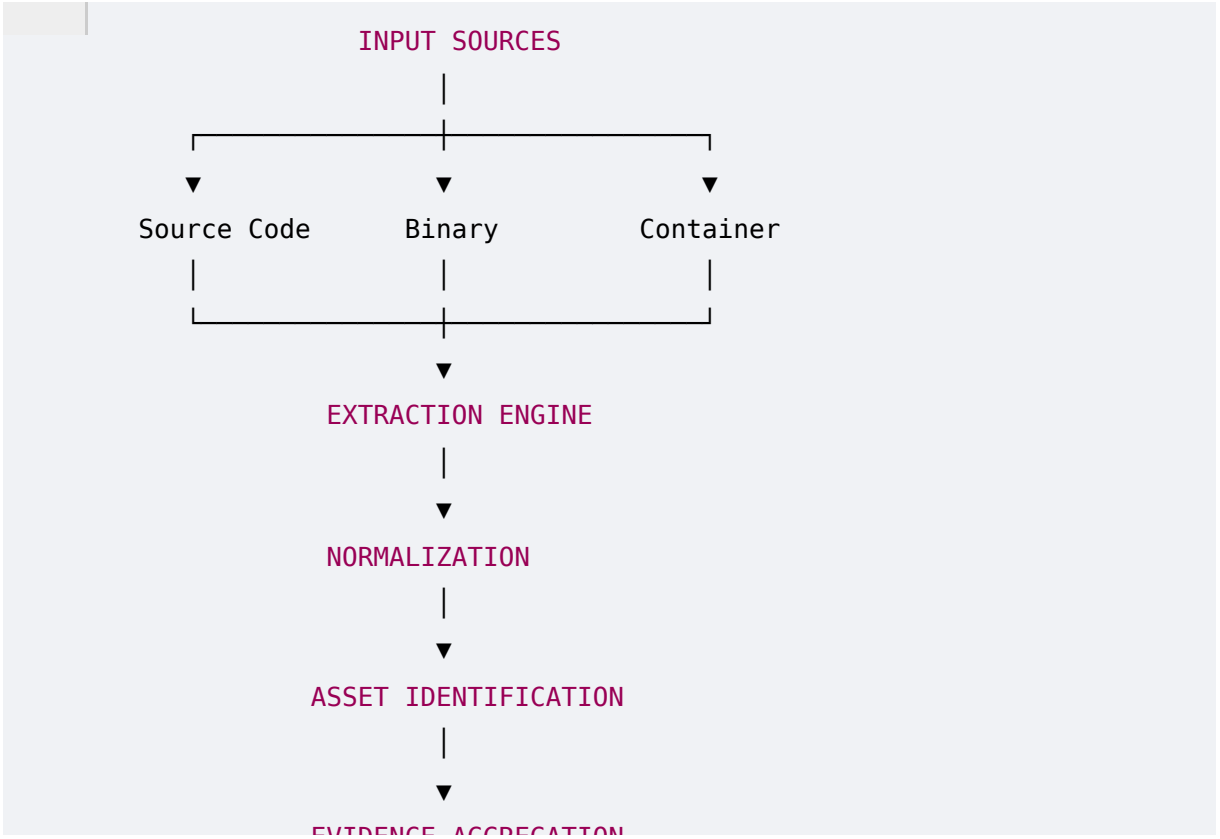
```
Highly Sensitive

Retention:
25 years
```

This dramatically improves risk analysis.

---

51. CBOM Generation Pipeline

The complete pipeline:



---

52. Example CBOM

A simplified ECDAT CBOM:

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.x",
  "metadata": {
    "component": "payments-api"
  },
  "cryptographicAssets": [
    {
      "id": "CRYPTO-001",
```

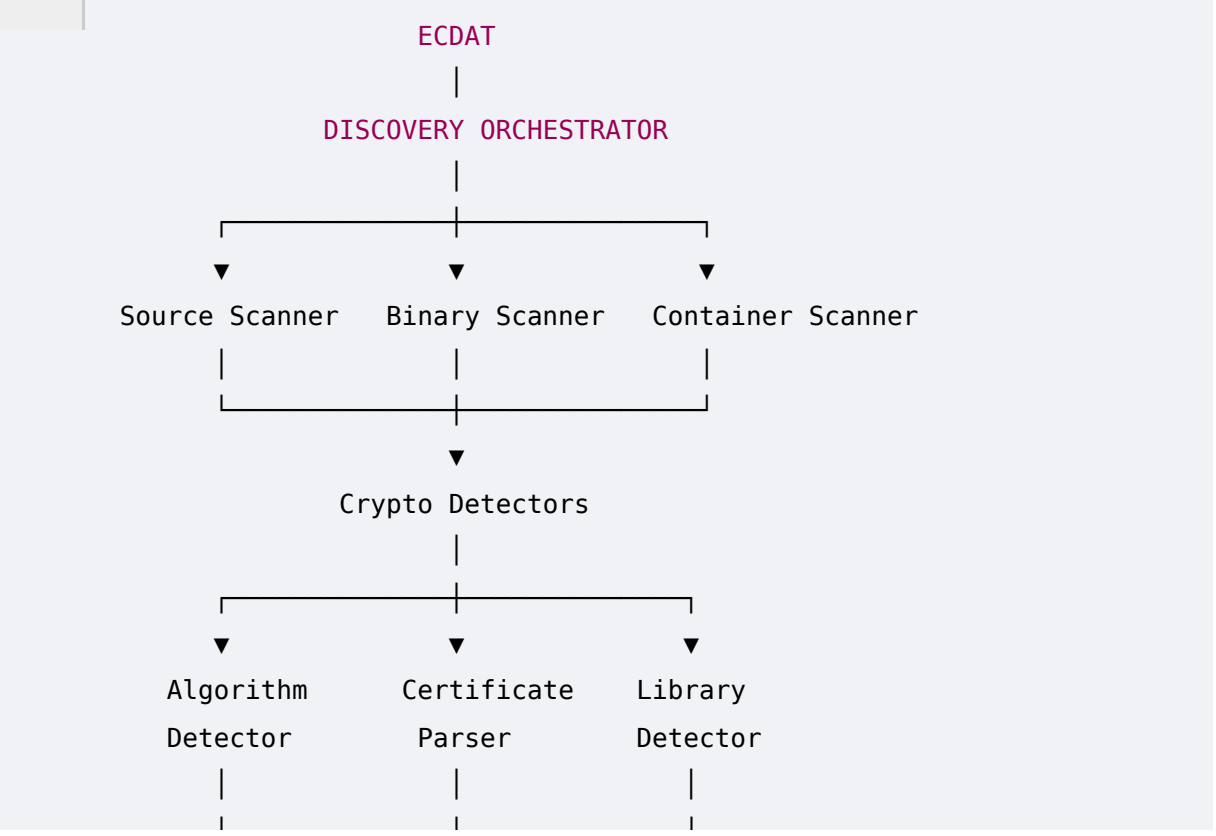
```
            "type": "algorithm",
            "algorithm": "ECDSA",
            "parameterSet": "P-256",
            "function": "signature",
            "location": "tls/server.crt",
            "quantumStatus": "vulnerable",
            "confidence": 0.99
          },
```

The production schema should follow the selected CBOM specification and preserve provenance/evidence.

---

53. Discovery Architecture

ECDAT's discovery subsystem could be structured as:



---

54. Continuous Discovery

Enterprise environments change constantly.

A repository changes.

A container changes.

A certificate expires.

A library updates.

A new service appears.

Therefore:

> ECDAT should not be a one-time scanner.

It should support continuous discovery.

```
Scan
 ↓
Detect Change
 ↓
Re-analyze
 ↓
Update CBOM
 ↓
Recalculate Risk
 ↓
Notify
```

---

55. CI/CD Integration

ECDAT can run during builds.

Example:

```
Developer Commit
        ↓
CI Pipeline
        ↓
ECDAT Scan
        ↓
Crypto Findings
        ↓
Risk Evaluation
        ↓
Policy Check
        ↓
Build Pass / Review / Fail
```

Example policy:

```
Reject deployment if:

New critical quantum-vulnerable
cryptographic dependency introduced.
```

This turns ECDAT into a preventive control.

---

56. Git Integration

ECDAT should understand version history.

Example:

```
Commit A
↓
RSA introduced

Commit B
↓
RSA usage modified

Commit C
↓
PQC migration
```

This allows:

- Crypto introduction tracking
- Crypto removal tracking
- Developer attribution
- Migration history

---

57. Incremental Scanning

Scanning an entire enterprise repeatedly is expensive.

Instead:

```
Initial Scan
↓
Full Inventory
```

Then:

```
New Commit
↓
Changed Files Only
↓
Incremental Scan
```

Similarly:

```
New Container Digest
↓
Scan New Layers
```

This improves performance.

---

58. Scan Scheduling

Possible modes:

On Demand

Security administrator starts scan.

Scheduled

```
Daily
Weekly
Monthly
```

Event Driven

```
New Git Commit
New Container
Certificate Renewal
Cloud Configuration Change
```

Continuous

Runtime monitoring detects changes.

---

59. Security of the Scanner

ECDAT itself becomes a highly privileged security platform.

Therefore, it must be designed securely.

Requirements include:

- Least privilege
- Strong authentication
- Encryption in transit
- Encryption at rest
- Audit logs
- Access controls
- Tenant isolation
- Secure credential handling
- Secret redaction

The scanner must not introduce a new security problem while attempting to discover security assets.

---

60. Privacy and Secret Handling

ECDAT may encounter:

```
Private Keys
Passwords
API Tokens
Certificates
Secrets
```

The default principle should be:

> **Discover metadata, not secret material.**

For example:

Bad:

```
private_key:
-----BEGIN PRIVATE KEY-----
...
```

Better:

```
Private Key Detected:
YES

Algorithm:
```

```
RSA

Location:
Vault Reference

Material:
NOT COLLECTED
```

This is an important design decision.

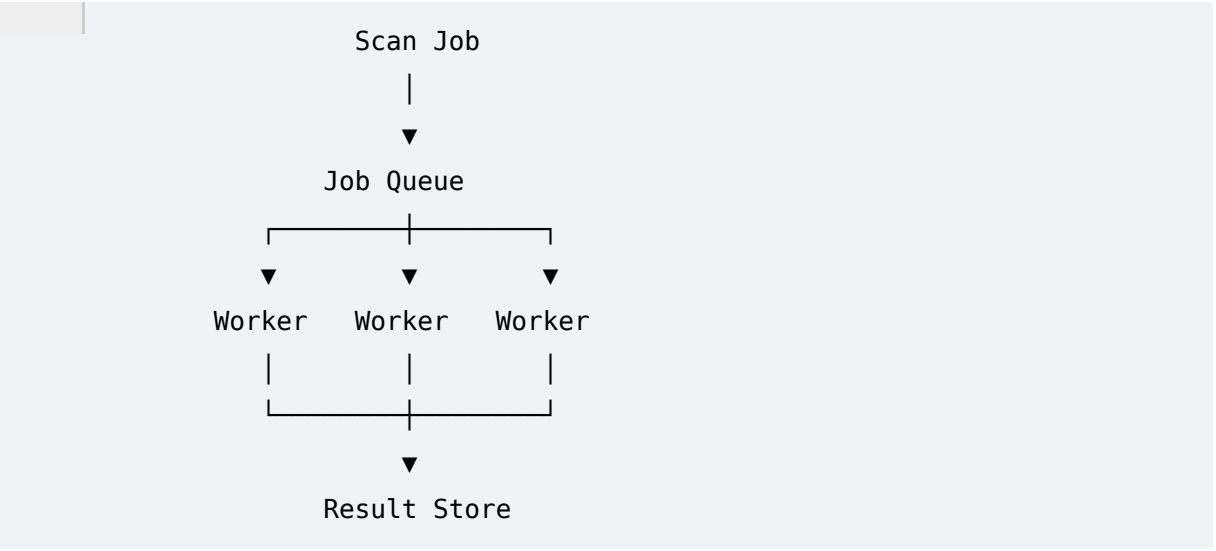
---

61. Performance and Scalability

Enterprise environments may contain millions of files.

Therefore, scanning must be scalable.

Potential architecture:



Technologies might eventually include:

- Redis
- Kafka
- RabbitMQ
- Celery
- Kubernetes Jobs

The exact choice belongs to the architecture phase.

---

62. Discovery Metrics

ECDAT should measure:

Coverage

```
Repositories Scanned
Containers Scanned
Binaries Scanned
Certificates Discovered
```

Detection

```
Crypto Assets
Algorithms
Libraries
Protocols
```

Quality

```
High Confidence
Medium Confidence
Low Confidence
```

Performance

```
Files/Second
Repositories/Hour
Scan Duration
Worker Utilization
```

---

63. ECDAT Discovery Dashboard

A dashboard might display:

```
┌─────────────────────────────────────────────┐
│            CRYPTOGRAPHIC INVENTORY          │
├──────────────┬──────────────────┬───────────┤
│ Assets       │ Algorithms       │ Certificates │
│ 184,231      │ 42               │ 71,291       │
├──────────────┼──────────────────┼──────────────┤
│ Libraries    │ Protocols        │ Unknown      │
│ 3,829        │ 17               │ 2,913        │
└──────────────┴──────────────────┴──────────────┘
```

Then:

```
Quantum Vulnerable
████████████████

PQC Ready
████████

Unknown
████
```

The UI is not merely aesthetic.

It gives decision-makers a view of enterprise cryptographic posture.

---

64. Example Enterprise Scan

Suppose ECDAT scans:

```
1,200 repositories
8,000 containers
3,000 binaries
20,000 certificates
```

Results:

```
Cryptographic Assets:
84,221

Unique Algorithms:
37

Quantum-Vulnerable:
31,402

Deprecated:
7,821

Unknown:
1,302
```

ECDAT then identifies:

```
Critical Quantum Assets:
2,817
```

The risk engine can later prioritize them.

---

65. Requirements Derived From This Phase

Phase 5 produces several core requirements.

Discovery

- Multi-source scanning
- Multi-language support
- Binary inspection
- Container inspection
- Certificate parsing
- Configuration analysis
- Dependency analysis

Intelligence

- Algorithm normalization
- Asset deduplication
- Evidence collection
- Confidence scoring
- Ownership mapping
- Data classification

Inventory

- CBOM generation
- Version history
- Dependency graph
- Asset lineage

Operations

- Incremental scanning
- CI/CD integration

- Git integration
- Scheduled scanning
- Continuous discovery

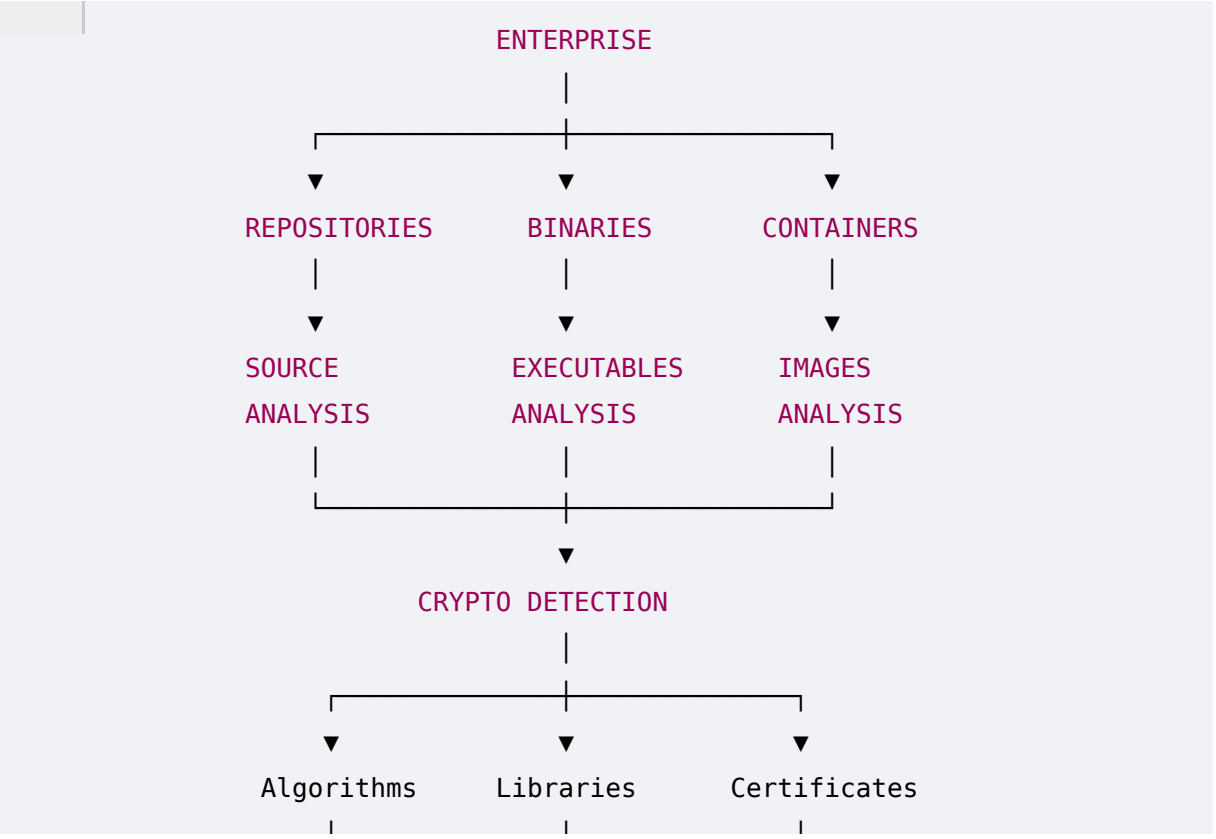
Security

- Least privilege
- Secret redaction
- Auditability
- Secure storage

---

66. Phase 5 — The Complete Discovery Model

At this point, ECDAT can be conceptualized as:



---

67. The Most Important Insight

A cryptographic inventory should never simply be a list.

This:

```
RSA
AES
```

```
ECC
SHA
```

is not enough.

ECDAT needs:

```
Algorithm
      +
Usage
      +
Location
      +
Application
      +
Library
      +
Protocol
      +
Data
      +
Owner
      +
Evidence
```

That turns raw discovery into **cryptographic intelligence**.

---

68. The Discovery-to-Decision Pipeline

The complete ECDAT journey now looks like:

```
                    RAW ENTERPRISE
                          |
                          ▼
                     DISCOVERY
                          |
                          ▼
                CRYPTO EXTRACTION
                          |
                          ▼
                ASSET NORMALIZATION
                          |
                          ▼
```

```
                    CBOM CREATION
                          |
                          ▼
                  DEPENDENCY GRAPH
                          |
```

This pipeline will become the backbone of the final architecture.

---

69. Phase 5 Summary

The central purpose of ECDAT is now becoming concrete.

The system must transform an organization's extremely fragmented cryptographic landscape into a structured, explainable, continuously updated inventory.

The key components are:

Discovery

Find cryptographic assets.

Normalization

Convert different findings into common representations.

Evidence

Record why the system believes an asset exists.

Confidence

Quantify uncertainty.

Deduplication

Combine multiple observations of the same underlying asset.

CBOM

Produce a machine-readable cryptographic inventory.

Dependency Graph

Understand relationships between applications, libraries, protocols, algorithms, keys, certificates, and data.

Continuous Monitoring

Detect changes over time.

---

70. Final Takeaway

The most important principle from Phase 5 is:

> **You cannot perform quantum-risk analysis on an enterprise that does not know what cryptography it is using.**

Therefore:

```
Discovery
        ↓
is the foundation of
        ↓
CBOM
        ↓
which enables
        ↓
Quantum Risk Assessment
        ↓
which enables
        ↓
PQC Migration Planning
```

This is why the NTRO requirement to scan **source repositories, binaries, libraries, and container images** is so important.

Those aren't arbitrary inputs.

They represent different layers of the enterprise software supply chain where cryptography can hide.

---

71. Phase 6 Preview — Mosca's Theorem & Quantum Risk Assessment

Now we have:

Phase 1: Cryptography

Phase 2: Quantum Threat

Phase 3: PQC

Phase 4: Enterprise Crypto Ecosystem

Phase 5: Discovery + CBOM

The next question is:

> **Once ECDAT discovers 100,000 cryptographic assets, how does it determine which ones actually matter?**

That takes us directly into the risk engine.

Phase 6 will deeply develop:

- Mosca's theorem
- `X + Y > Z`
- Data lifetime
- Migration time
- Quantum threat horizon
- HNDL
- Business criticality
- Data sensitivity
- Exposure
- Algorithm vulnerability
- Migration complexity
- Risk scoring
- Risk normalization
- Confidence-adjusted risk
- Risk matrices
- Prioritization
- Scenario analysis
- Quantum threat assumptions
- Best-case / worst-case models
- Enterprise risk dashboards
- Risk propagation through dependency graphs
- Critical-path analysis
- "What breaks if this crypto breaks?"
- Migration ROI
- Executive risk reporting
- A concrete ECDAT Quantum Risk Score model

Most importantly, Phase 6 will turn the inventory we designed here into a **mathematically defensible quantum-risk engine** rather than an arbitrary red/yellow/green dashboard.

Phase 5 complete.